Slide Docente – Modulo Pratico: Git, Unsloth Studio e OpenCode per la Digitalizzazione Industriale

Ente: Fondazione ITS MAKER – Sede di Bologna

Durata: 6 Ore (Core: Blocchi 1–6) + 2 Ore (Estensione: Blocchi 7–8)

Formato Didattico: 20% Inquadramento al Proiettore / 80% Laboratorio su VM

BLOCCO 1 (Ora 1) – L'Ecosistema Open Source e Git come "Salvataggio Partita"

Slide 1.1 – Benvenuti nell'Officina Digitale

Dal pezzo meccanico al software di reparto

- **In officina non costruisci una vite M8 dal pieno:** apri il catalogo, leggi la scheda tecnica e la prelevi dal magazzino.
- **Nel software industriale funziona uguale:**
- **Open Source (GitHub):** il magazzino mondiale dei semilavorati software gratuiti e ispezionabili.
- **Git:** la macchina del tempo che salva ogni lavorazione buona e ti riporta indietro se sbagli.
- **Unsloth Studio:** il banco prova motori dove facciamo girare l'Intelligenza Artificiale in locale (senza mandare dati riservati all'esterno).
- **OpenCode:** l'operatore robotico a cui dettiamo in italiano cosa costruire (*Vibe Coding*).

> **Regia Docente (3 min):** Rompere subito il ghiaccio: *"Oggi non faremo un esame di programmazione a memoria. Impareremo a dirigere un'officina digitale: scegliere i pezzi giusti, testare il motore AI sulla nostra macchina e far scrivere il codice a un agente senza mai perdere il lavoro."*

Slide 1.2 – Come si legge un Progetto Open Source in 60 Secondi

Il file `README.md` è la scheda tecnica della macchina

Quando cerchi uno strumento su GitHub (es. un lettore di file CSV, un cruscotto di manutenzione, un calcolatore di tempi), guarda sempre **4 indicatori**:

Indicatore su GitHub	Cosa significa in Officina	Segnale Verde (Affidabile)	Segnale Rosso (Scartare)
`README.md`	Manuale d'uso e montaggio	Spiega cosa fa, ha screenshot e istruzioni chiare	Assente o di 2 righe incomprensibili
Ultimo Commit	Ultima manutenzione effettuata	Pochi giorni/mesi fa (o tool stabile e completo)	7 anni fa con 50 errori aperti
Stars / Issues	Quante fabbriche lo usano / Segnalazioni guasti	Buon numero di stelle, problemi risolti	Errori bloccanti senza risposta
Licenza (MIT / Apache)	Permesso di utilizzo in azienda	Uso commerciale e modifica consentiti	Nessuna licenza o divieto d'uso

> **Regia Docente (5 min):** Aprire al proiettore un repository reale (es. `github.com/unslothai/unsloth` o il repository template della lezione) e indicare con il mouse dove si trovano `README.md`, *Commits*, *Issues* e *License*.

Slide 1.3 – Perché nasce Git: Basta con `Scheda_ Finale_v3_VERA_ULTIMA.xlsx`!

Il problema dei file duplicati in Ufficio Tecnico

- Chi non usa Git salva i file così:
- `manutenzione.html`
- `manutenzione_nuova.html`
- `manutenzione_ok_giovedi.html`
- `manutenzione_NON_TOCCARE_DEFINITIVA.html`
- **Risultato:** dopo due ore (specialmente quando usi l'AI per modificare i file!) non sai più quale file funziona e quale ha il calcolo rotto.
- **Con Git hai UN SOLO FILE nella cartella**, ma dietro c'è una **linea del tempo invisibile** con tutti i punti di salvataggio certificati (*Commit*).

> **Regia Docente (3 min):** Chiedere per alzata di mano chi ha mai chiamato una tesina o un disegno CAD `definitivo_ultimo_2`. Spiegare che nel *Vibe Coding* l'AI modifica 200 righe in 3 secondi: senza Git, se il terzo comando rompe tutto, bisogna buttare via il lavoro.

Slide 1.4 – I 4 Comandi Git dell'Officina (Gli unici che vi servono oggi)

Dal Magazzino al Fungo di Emergenza

```
# 1. PRELEVA DAL MAGAZZINO (Scarica una copia completa del progetto sulla tua VM)

git clone <indirizzo-progetto>



# 2. ISPEZIONA IL BANCO (Controlla quali file hai modificato dall'ultimo salvataggio)

git status



# 3. METTI IN CASSAFORTE (Prepara il pezzo e registra il punto di salvataggio con un'etichetta)

git add .

git commit -m "v1: tabella manutenzione funzionante"



# 4. FUNGO DI EMERGENZA! (Hai rotto tutto? Torna istantaneamente all'ultimo commit buono)

git restore .
```

> **Regia Docente (4 min):** Mostrare i 4 comandi in diretta al proiettore in meno di 2 minuti. Sottolineare il comando `git restore .`: *"Questo è il vostro paracadute quando tra poco useremo l'AI."* Passare subito la parola agli studenti per il **Laboratorio 1 (45 min)**.

BLOCCO 2 (Ora 2) – Dentro il Motore AI: Unsloth Studio e Modelli GGUF

Slide 2.1 – Perché far girare l'AI sulla propria Macchina (e non solo su ChatGPT)?

Tre motivi concreti per le aziende manifatturiere della Motor Valley / Packaging Valley

1. **Segreto Industriale e Privacy:**

- Non puoi incollare distinte base riservate, costi fornitore, disegni o log di produzione di un cliente su un sito pubblico gratuito. Un modello locale su **Unsloth Studio** gira **100% offline** dentro il perimetro aziendale.

1. **Continuità Operativa in Reparto (Zero Internet):**

- A bordo macchina o in un capannone schermato il Wi-Fi può cadere. Il modello locale sul PC di reparto continua a rispondere.

1. **Zero Costi a Consumo:**

- Nessun abbonamento mensile per operatore: usi l'hardware che hai già.

> **Regia Docente (5 min):** Collegare il tema alla realtà delle PMI meccaniche e meccatroniche bolognesi: gli uffici tecnici vogliono usare l'AI, ma i titolari bloccano tutto per paura che escano listini e codici ricambi.

Slide 2.2 – Il Problema Hardware: Come entra un Motore AI in una VM senza Scheda Video?

Parametri (Miliardi) e Quantizzazione (GGUF) spiegati senza matematica

- **Cosa sono i Parametri (`0.6B`, `1.5B`, `3B`, `70B`)?**
- Sono i "neuroni/ingranaggi" del modello. `1.5B` = 1,5 Miliardi di parametri.
- Più parametri = più cultura e ragionamento, ma **più RAM e più lentezza**.
- **Cos'è la Quantizzazione (`GGUF` a 4-bit, 2-bit, 1.58-bit)?**
- È come comprimere un assieme CAD 3D da 500 MB in un file STEP/PDF leggero da 25 MB:
- **16-bit (Originale):** precisione millimetrica al centesimo di micron, ma pesa **3 GB** ogni miliardo di parametri (serve una GPU costosa).
- **4-bit (`Q4_K_M`):** ottimo compromesso! Pesa **75% in meno** (~0.9 GB per 1.5B parametri) e mantiene quasi tutta l'intelligenza.
- **1.58-bit / 2-bit (`UD-IQ2_M`):** compressione estrema! Entra ovunque (450 MB), corre velocissimo su CPU, ma inizia a "perdere colpi" sui calcoli precisi.

> **Regia Docente (5 min):** Usare il paragone delle tolleranze meccaniche: *"Per fare un'etichetta o una bozza non ti serve la rettifica al micron (16-bit), ti basta una lavorazione standard a 4-bit che pesa un quarto. Ma se scendi a 1.58-bit (sgrossatura pesante), il pezzo entra ovunque ma balla nelle sedi."*

Slide 2.3 – Il Cruscotto di Unsloth Studio (`localhost:8888`)

Leggere i semafori hardware prima di avviare il motore

```
# Avvio del server web locale di Unsloth Studio

unsloth studio -p 8888
```

Quando scegliete un modello GGUF dentro Unsloth Studio, il sistema analizza la vostra VM e mostra un badge:

- **RECOMMENDED (Verde):** il modello entra comodamente nella RAM della VM e girerà fluido.
- **TIGHT (Giallo):** entra al pelo nella memoria; chiudete le schede browser inutili.
- **OOM – Out Of Memory (Rosso):** troppo pesante per questa macchina! Se lo caricate, la VM si blocca.

> **Regia Docente (5 min):** Mostrare al proiettore l'apertura di `http://localhost:8888`, far vedere dove sono elencati i modelli GGUF pre-caricati nella VM e lanciare il **Laboratorio 2A (45 min)** di misurazione RAM e Token/s.

BLOCCO 3 (Ora 3) – Stress Test Industriale nella Model Arena

Slide 3.1 – Il Trilemma dell'AI Locale in Officina

Peso (RAM) vs Velocità (Token/s) vs Precisione (Allucinazioni)

Taglia & Quantizzazione	Velocità su CPU VM	Consumo RAM	Risultato sui Compiti di Reparto
`0.6B` / `1B` a 2-bit (`UD-IQ2_M`)	**22–28 token/s** (Istantaneo)	**< 850 MB** (Leggerissimo)	Riassume testi brevi, ma **sbaglia le sottrazioni** (es. `78°C - 65°C`) o inventa codici ricambio (*allucinazione*).
`1.5B Coder` a 4-bit (`Q4_K_M`)	**12–15 token/s** (Fluido)	**~1.45 GB** (Ideale per VM)	**Punto di equilibrio (Sweet Spot):** capisce l'italiano tecnico, rispetta le tabelle e scrive ottimo codice HTML/JS/Python.
`3B Coder` a 3-bit (`UD-Q3_K_M`)	**5–8 token/s** (Lento su CPU)	**~2.2 GB** (Impegnativo)	Molto preciso sul codice complesso, ma se 24 VM lo usano insieme la CPU condivisa rallenta.

> **Regia Docente (10 min):** Mostrare dal vivo la scheda **Model Arena** di Unsloth Studio. Incollare la nota di fermo macchina della Pressa P03 e far vedere agli studenti come il modello ultra-compresso a 2-bit risponde per primo ma sbaglia il calcolo del superamento soglia termica, mentre il 4-bit risponde giusto. Avviare il **Laboratorio 2B (50 min)**.

BLOCCO 4 (Ora 4) – Dal Motore al Braccio Operativo: OpenCode e il Vibe Coding

Slide 4.1 – Cos'è il "Vibe Coding" (e perché cambia il lavoro tecnico)

Tu sei il Capo Reparto, OpenCode è l'Operatore che scrive il codice

- **Vecchio metodo (Copia-Incolla dalla Chat):**

1. Chiedi a una chat AI di scrivere un programma.
2. Copi il codice, apri un editor, incolli, salvi.
3. Al secondo giro non sai più *dove* incollare le modifiche e rompi il file.

- **Metodo Agente con OpenCode (`unsloth start opencode`):**

1. OpenCode vive **dentro la tua cartella di lavoro**.
2. Legge da solo i file esistenti, crea o modifica `manutenzione.html` direttamente su disco.
3. Tu tieni aperto il browser, premi **F5 (Aggiorna)**, collaudi visivamente il risultato e dai l'ordine successivo!

> **Regia Docente (8 min):** Mostrare la differenza tra Unsloth Chat (solo conversazione) e OpenCode (agente che tocca i file veri nella cartella).

Slide 4.2 – Come Collegare Unsloth Studio a OpenCode (Piano A e Piano B d'Aula)

Architettura Locale o Server di Reparto Condiviso

```
# PIANO A: Uso il motore AI caricato sulla MIA macchina virtuale

cd ~/mio-progetto-maker

unsloth start opencode
```

Cosa succede se tutte le 24 VM generano codice contemporaneamente e la CPU rallenta? Passiamo al Piano B (Server di Reparto LAN del Docente):

- In azienda non si mette un motore pesante su ogni tablet: si mette **un unico server AI in rete locale** e tutti i terminali di reparto si collegano a quello.
- In OpenCode basta digitare `/model` → `Connect provider` → `Custom`:
- **Base URL:** `http://<IP-DOCENTE>:8888/v1/`
- **API Key:** `sk-unsloth-aula`

> **Regia Docente (7 min):** Far lanciare `unsloth start opencode` agli studenti. Tenere già acceso sul PC docente `unsloth run --model unsloth/Qwen2.5-Coder-7B-Instruct-GGUF -H 0.0.0.0 --disable-tools -p 8888` pronto come turbo di backup per chi ha la VM lenta. Lanciare il **Laboratorio 3 – Fase v1 (45 min)**.

BLOCCO 5 (Ora 5) – Iterazione Rapida e Gestione del "Disastro Controllato"

Slide 5.1 – La Regola d'Oro del Vibe Coding Industriale

Mai fare il passo `v2` senza aver messo in cassaforte il passo `v1`!

Il ciclo di lavoro obbligatorio in 4 mosse:

1. **ORDINA (`OpenCode`):** Dai un prompt chiaro, specifico e un passo alla volta (es. *"Crea la tabella base con 6 controlli"*).
2. **COLLAUDA (`Browser`):** Apri il file `.html` nel browser e clicca tutti i pulsanti. Funziona?
3. **METTI IN CASSAFORTE (`Git`):**

```
git add .

git commit -m "v1: scheda base funzionante"
```

1. **RIPETI o RIPRISTINA:** Ora chiedi la `v2`. Se l'AI impazzisce e cancella metà pagina:

```
git restore .
```

In **1 secondo** sei tornato alla `v1` perfetta e puoi riformulare la richiesta!

> **Regia Docente (10 min):** Simulare dal vivo al proiettore un prompt sbagliato che rompe il file HTML. Mostrare `git diff` (le righe rosse cancellate dall'AI) e poi il salvataggio miracoloso con `git restore .`. Avviare il **Laboratorio 3 – Fasi v2 e v3 (50 min)**.

BLOCCO 6 (Ora 6) – Collaudo Finale, Documentazione `README.md` e Consegna

Slide 6.1 – Un Software senza Manuale non entra in Officina

Far scrivere a OpenCode il `README.md` e verificare la Storia Git

Un progetto professionale è completo solo quando un collega di un altro turno può aprirlo e capire subito come usarlo:

1. **Chiedi a OpenCode di documentare il lavoro:**

```
Crea un file README.md chiaro in italiano per i colleghi del reparto che spieghi:

1. Quale problema di officina risolve questo strumento.

2. Come si apre e utilizza passo dopo passo (senza bisogno di internet).

3. Elenco delle funzioni implementate nelle versioni v1, v2 e v3.
```

1. **Sigilla la consegna con Git e controlla il libretto tagliandi (`git log`):**

```
git add .

git commit -m "docs: aggiunto manuale operativo README.md"

git log --oneline
```

> **Regia Docente (10 min):** Mostrare come `git log --oneline` visualizza i 4 commit ordinati (`v1`, `v2`, `v3`, `docs`). Negli ultimi 20 minuti organizzare il **Collaudo Incrociato**: ogni studente prova l'app del vicino di banco cercando di metterla in crisi!

BLOCCHI 7 & 8 (Estensione 8 Ore) – Dati Sporchi CSV e Regole Aziendali

`AGENTS.md`

Slide 7.1 – [Ora 7] La Realtà delle Aziende: I Dati sono sempre "Sporchi"!

Ripulire esportazioni gestionali e log PLC con Unsloth + OpenCode

Quando esportate un file `.csv` da un vecchio gestionale AS/400, da Excel o da un PLC, trovate sempre:

- Punto e virgola (`;`) mescolati alle virgole (`,`).
- Prezzi scritti con la virgola (`12,50`) che bloccano i calcoli matematici (`NaN`).
- Codici articolo duplicati o righe vuote a metà tabella.

Come si risolve in Vibe Coding:

1. Carichiamo il file `distinta_sporca.csv` nella cartella del progetto.
2. Diciamo a OpenCode: *"Leggi `distinta_sporca.csv`, guarda come sono formattate le righe rotte e modifica la nostra web-app per accettare sia `;` sia `,`, convertire i decimali italiani e segnalare in rosso le anomalie."*

> **Regia Docente (10 min):** Mostrare il file `distinta_sporca.csv` reale in Blocco Note, far vedere gli errori nascosti e lanciare la sfida pratica da 50 minuti.

Slide 8.1 – [Ora 8] Standardizzare l'Agente: Il File `AGENTS.md`

Dare a OpenCode il "Regolamento di Fabbrica" una volta per tutte

Invece di ripetere in ogni prompt *"scrivi in italiano, non usare librerie internet, usa colori ad alto contrasto"*, creiamo nella cartella un file chiamato **`AGENTS.md`**. OpenCode lo legge automaticamente prima di ogni operazione!

```
# Regole di Reparto ITS MAKER (AGENTS.md)

1. Lingua: Tutte le etichette, i messaggi di errore e i commenti devono essere in ITALIANO tecnico chiaro.

2. Architettura Offline: Vietato usare CDN esterni (niente Bootstrap, Tailwind o Chart.js da internet). Tutto il codice HTML, CSS e JS deve funzionare 100% offline sulla VM.

3. Ergonomia Bordo Macchina: Pulsanti alti almeno 48px (usabili su schermi touch con i guanti), colori semaforici ISO (Verde #16a34a = OK, Giallo #eab308 = Attenzione, Rosso #dc2626 = Allarme/Fermo).

4. Sicurezza Dati: Non cancellare mai funzioni esistenti quando aggiungi una nuova richiesta.
```

> **Regia Docente (10 min):** Spiegare che `AGENTS.md` è il capitolato tecnico fisso dell'officina. Gli studenti creano il proprio `AGENTS.md`, lanciano un ultimo aggiornamento grafico/funzionale e presentano il progetto finale alla classe.